Experiment 6

Student Name: Aashish Pant **UID: 22BCS12862**
Branch: CSE **Section: IoT-642-B**
Semester: 6 **Date of Performance:03/03/**
Subject Name: Java with Lab **Subject Code: 22CSH-359**

1. Aim:
Develop Java programs using lambda expressions and stream operations for sorting, filtering, and processing large datasets efficiently.

2. Objective:

• Develop Java programs using lambda expressions and stream operations for sorting, filtering, and processing large datasets efficiently.

• Implement easy, medium, and hard-level tasks involving sorting employees, filtering and sorting students, and processing products using streams.

3. Implementation/Code:

a.

```
import java.util.*;
import java.util.stream.Collectors;

class Employee {
    String name;
    int age;
    double salary;

    public Employee(String name, int age, double salary) {
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    public void display() {
        System.out.println(name + " (" + age + ", " + salary + ")");
    }
}

public class EmployeeSorting {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
```

```
        employees.add(new Employee("Alice", 30, 50000));
        employees.add(new Employee("Bob", 25, 60000));
        employees.add(new Employee("Charlie", 35, 55000));
        employees.add(new Employee("Alex", 28, 45000));
        employees.add(new Employee("Alex", 32, 47000));
        employees.add(new Employee("Alex", 25, 46000));
        employees.add(new Employee("David", 29, 50000));
        employees.add(new Employee("Eve", 31, 50000));
        employees.add(new Employee("Frank", 27, 50000));

        // Sorting by Name (Alphabetical Order)
        List<Employee> sortedByName = employees.stream()
                .sorted(Comparator.comparing(e -> e.name))
                .collect(Collectors.toList());

        System.out.println("Sorted by Name:");
        sortedByName.forEach(Employee::display);

        // Sorting by Age (Ascending Order)
        List<Employee> sortedByAge = employees.stream()
                .sorted(Comparator.comparingInt(e -> e.age))
                .collect(Collectors.toList());

        System.out.println("\nSorted by Age:");
        sortedByAge.forEach(Employee::display);

        // Sorting by Salary (Descending Order)
        List<Employee> sortedBySalary = employees.stream()
                .sorted(Comparator.comparingDouble((Employee e) -> e.salary).reversed())
                .collect(Collectors.toList());

        System.out.println("\nSorted by Salary:");
        sortedBySalary.forEach(Employee::display);
    }
     }
```

b.

```
import java.util.*;
import java.util.stream.Collectors;

class Student {
    String name;
    double marks;

    public Student(String name, double marks) {
        this.name = name;
```

```
        this.marks = marks;
    }

    public void display() {
        System.out.println(name + " (" + marks + ")");
    }
}

public class StudentFilterSort {
    public static void main(String[] args) {
        List<Student> students = Arrays.asList(
            new Student("Alice", 80),
            new Student("Bob", 72),
            new Student("Charlie", 90),
            new Student("David", 65),
            new Student("Eve", 85),
            new Student("Frank", 65)
        );

        // Filter students with marks > 75, sort by marks (descending), then by name
        List<Student> filteredSortedStudents = students.stream()
            .filter(s -> s.marks > 75)
            .sorted(Comparator.comparingDouble((Student s) -> s.marks).reversed()
                    .thenComparing(s -> s.name))
            .collect(Collectors.toList());

        // Display results
        if (filteredSortedStudents.isEmpty()) {
            System.out.println("No students scored above 75%");
        } else {
            filteredSortedStudents.forEach(Student::display);
        }
    }
}
```

c.

```
import java.util.*;
import java.util.stream.Collectors;

class Product {
    String name;
    String category;
    double price;

    public Product(String name, String category, double price) {
```

```
        this.name = name;
        this.category = category;
        this.price = price;
    }

    @Override
    public String toString() {
        return name + " ($" + price + ")";
    }
}

public class ProductProcessor {
    public static void main(String[] args) {
        List<Product> products = Arrays.asList(
            new Product("Laptop", "Electronics", 1200),
            new Product("Phone", "Electronics", 800),
            new Product("TV", "Electronics", 1500),
            new Product("Jeans", "Clothing", 60),
            new Product("T-Shirt", "Clothing", 25),
            new Product("Sneakers", "Footwear", 90),
            new Product("Boots", "Footwear", 120),
            new Product("Sandals", "Footwear", 120)
        );

        // Group products by category
        Map<String, List<Product>> groupedByCategory = products.stream()
                .collect(Collectors.groupingBy(p -> p.category));

        // Find most expensive product in each category
        Map<String, Optional<Product>> mostExpensiveByCategory = products.stream()
                .collect(Collectors.groupingBy(
                        p -> p.category,
                        Collectors.maxBy(Comparator.comparingDouble(p -> p.price))
                ));

        // Calculate the average price of all products
        double averagePrice = products.stream()
                .collect(Collectors.averagingDouble(p -> p.price));

        // Display grouped products
        System.out.println("Grouped Products:");
        groupedByCategory.forEach((category, productList) -> {
            System.out.println(category + ": " + productList);
        });

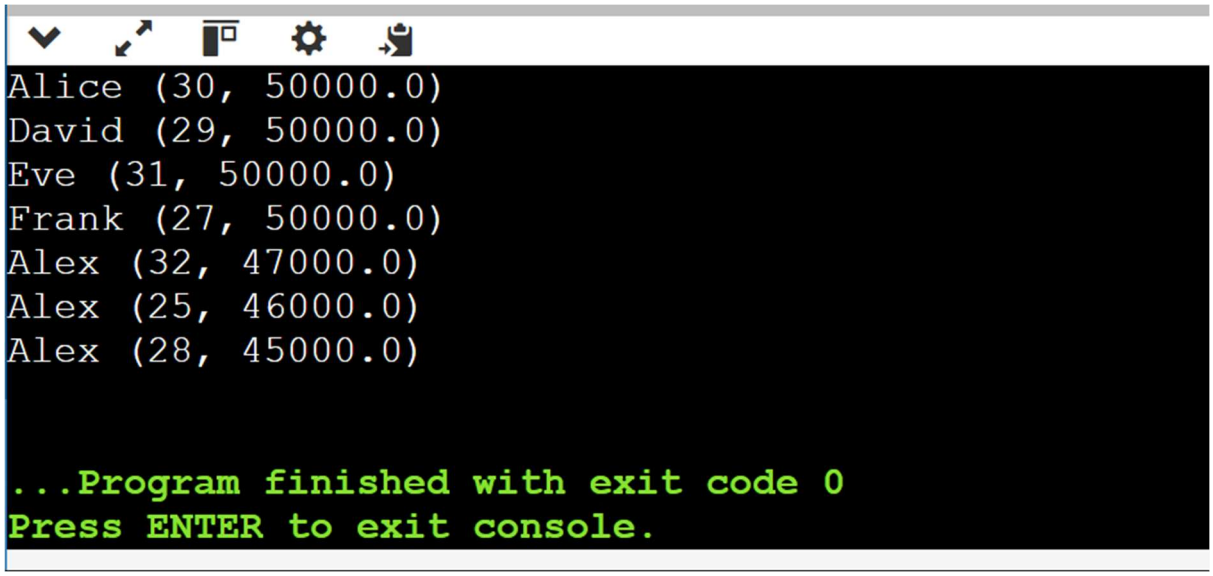
        // Display most expensive product in each category
```

```
        System.out.println("\nMost Expensive Product per Category:");
        mostExpensiveByCategory.forEach((category, product) ->
            System.out.println(category + ": " + product.orElse(null))
        );

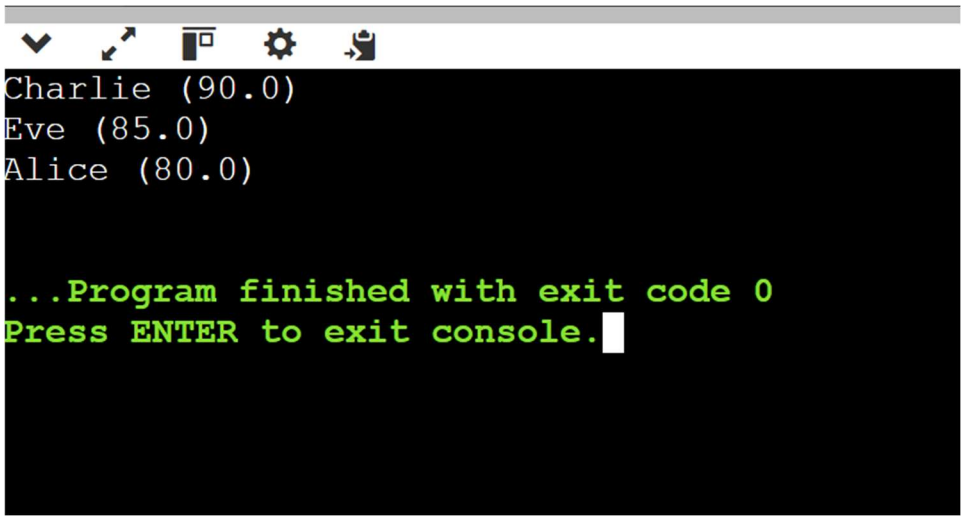
        // Display average price
        System.out.println("\nAverage Price of All Products: $" + averagePrice);
    }
}
```

4. **Output:**

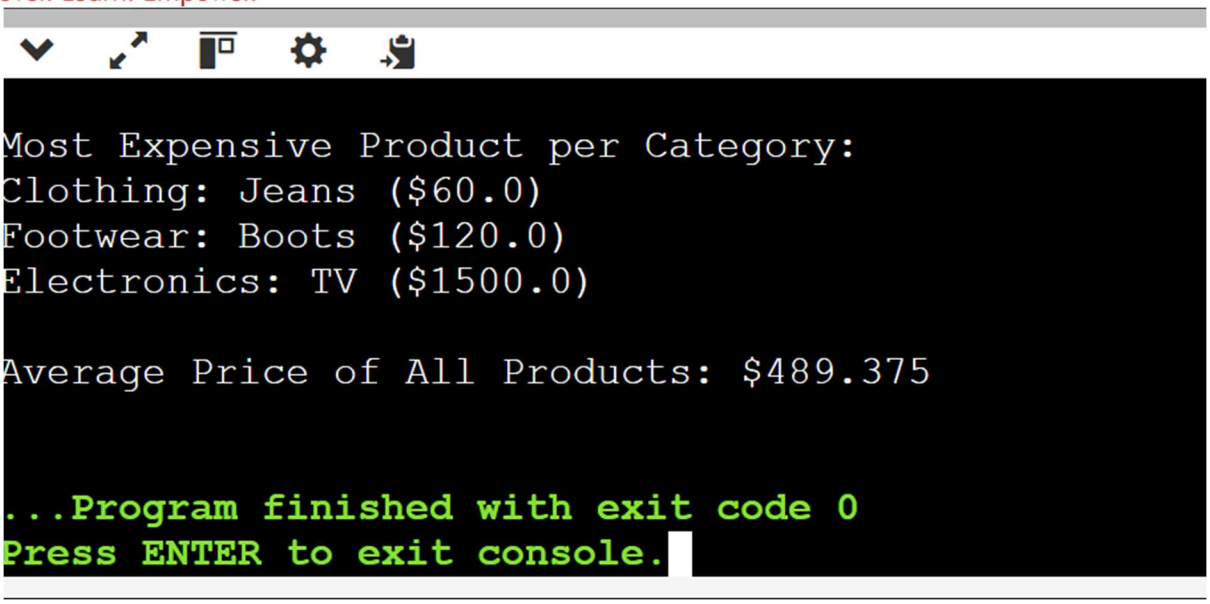
a.



b.



c.



5. Learning Outcome:

- Understand and implement **lambda expressions** for sorting objects in a list based on different attributes.
- Utilize **Java Streams API** to perform operations like **filtering, sorting, and mapping** efficiently on large datasets.
- Learn **Comparator and method references** to simplify object comparisons for sorting.
- Apply **grouping and aggregation functions** using Collectors.groupingBy() and Collectors.maxBy() for processing categorized data.
- Gain hands-on experience in computing **statistical values** like the **average** from a dataset using mapToDouble() and average().
- Improve **code efficiency and readability** by using **functional programming** techniques in Java.